

PROGRAMACIÓN / IA — NIVEL 3 DE 3

Programar con IA: proyectos completos

Cómo llevar un proyecto real de punta a punta usando IA en cada etapa: arquitectura, construcción, testing, deploy y mantenimiento en producción.

Arquitectura

Deploy

Producción

Índice

- 01 Introducción — de tareas puntuales a un proyecto completo
- 02 Planificar antes de programar: arquitectura asistida por IA
- 03 Construyendo el proyecto: backend, frontend y base de datos
- 04 Testing y calidad con IA
- 05 Deploy a producción
- 06 Mantenimiento: monitoreo, logs e iteración
- 07 Errores comunes en proyectos reales con IA
- 08 Checklist final

01 · Introducción

Este es el cierre de la serie de Programación/IA de UpgradeLab. Si en el Nivel 1 construiste algo desde cero sin saber programar, y en el Nivel 2 aprendiste a integrar IA a tareas puntuales dentro de un proyecto ya existente, este Nivel 3 conecta todo: cómo llevar un proyecto real de punta a punta.

La diferencia con los niveles anteriores no es la herramienta — es la **secuencia**. Un proyecto completo no es una serie de prompts sueltos: es planificación, construcción por etapas, verificación constante, y decisiones que hay que sostener en el tiempo, mucho después de que la IA generó el primer borrador.

QUÉ VAS A PODER HACER AL TERMINAR ESTA GUÍA

Planificar la arquitectura de un proyecto antes de escribir la primera línea, construirlo por etapas verificables, usar IA para testing y control de calidad, hacer un deploy real a producción, y mantenerlo funcionando con monitoreo básico una vez que tiene usuarios reales.

02 · Planificar antes de programar

El error más común en proyectos asistidos por IA es empezar a construir antes de decidir qué se va a construir.

Antes de escribir código, conviene definir con la IA — como si fuera una sesión de diseño, no de programación:

1. **Qué problema resuelve el proyecto** y para quién.
2. **Las piezas principales:** qué necesita guardar (base de datos), qué necesita mostrar (interfaz), con qué servicios externos se conecta (pagos, emails, storage).
3. **El alcance de la primera versión** — qué queda afuera a propósito para no eternizar el lanzamiento.

PROMPT DE PLANIFICACIÓN DE EJEMPLO

Quiero construir [tu proyecto]. Los usuarios van a poder [funcion principal]. Antes de escribir código, ayudame a definir: que tablas necesita la base de datos, que paginas necesita el frontend, y que hay que dejar fuera de la primera version para lanzar rapido. Dame esto como un plan, no como código todavía.

POR QUE CONVIENE ESTE PASO

Un plan claro, aunque sea breve, evita que la IA tome decisiones de arquitectura por vos sin que te des cuenta — y te da un documento de referencia para todo el resto del proyecto.

03 · Construyendo el proyecto

Con el plan definido, construir por etapas es mucho más manejable que pedir "todo el proyecto" de una vez.

1

Base de datos primero

Definí las tablas y sus relaciones antes que nada — todo lo demás depende de esta estructura, y cambiarla después de construir encima es más costoso.

2

Backend / lógica central

Las funciones que hacen el trabajo real: crear, leer, actualizar datos, validar reglas de negocio.

3

Frontend / interfaz

Una vez que la lógica funciona, construir la interfaz sobre algo que ya se puede probar es mucho más simple que construir ambos a ciegas al mismo tiempo.

4

Integraciones externas al final

Pagos, emails, servicios de terceros: conviene integrarlos cuando el resto ya funciona, para poder distinguir un error propio de un error de la integración.

Ojo: pedirle a la IA "construime toda la app" en un solo prompt gigante suele generar algo que funciona a medias y es difícil de depurar, porque no hay forma de saber en que paso se rompió algo. Ir por etapas, verificando cada una, ahorra horas mas adelante.

04 · Testing y calidad con IA

Un proyecto completo necesita alguna forma de confirmar que sigue funcionando a medida que crece.

- **Probá cada etapa antes de seguir a la siguiente.** No avances a construir el frontend si el backend todavía no responde lo que debería.
- **Pedile a la IA que genere casos de prueba para los flujos críticos** — registro, pago, cualquier acción que si falla, falla en serio.
- **Probá los casos límite**, no solo el camino feliz: qué pasa si un campo viene vacío, si un pago se rechaza, si dos personas hacen la misma acción al mismo tiempo.

PROMPT DE EJEMPLO

Este es el flujo de registro de usuarios [pegar código o descripción]. Decime que casos límite no estoy contemplando y por qué cada uno importa.

05 · Deploy a producción

Pasar de "funciona en mi computadora" a "funciona en internet, para cualquiera" tiene sus propios cuidados.

- **Variables de entorno separadas.** Las claves y configuraciones de producción nunca deberían ser las mismas que usás mientras probás en local.
- **Un dominio de prueba antes que el real.** La mayoría de las plataformas de deploy (Vercel, Netlify, Railway) te dan una URL temporaria para verificar todo antes de conectar tu dominio definitivo.
- **Revisá los logs después del primer deploy real.** Es el momento en el que suelen aparecer errores que no se veían en local (variables faltantes, permisos, límites de la plataforma).

PROMPT UTIL PARA DIAGNOSTICAR ERRORES DE DEPLOY

```
[Pegar el error exacto del log de produccion]. Este es el codigo relacionado:  
[pegar archivo]. Explicame la causa mas probable antes de proponer un cambio.
```

06 · Mantenimiento

Un proyecto en producción con usuarios reales necesita atención continua, no solo el esfuerzo inicial de construirlo.

- **Revisá los logs periódicamente**, no solo cuando algo se rompe visiblemente — muchos errores quedan silenciosos si nadie los reporta.
- **Guardá un registro de qué cambia y cuándo**. Cuando algo falla, poder preguntar "qué cambié en los últimos días" ahorra mucho tiempo de diagnóstico.
- **Usá la IA para analizar patrones en errores recurrentes**, pegándole logs reales en vez de descripciones vagas del problema.

07 · Errores comunes en proyectos reales con IA

- **Construir sin plan.** Ya visto en el capítulo 2 — es la causa más frecuente de proyectos que hay que rehacer parcialmente.
- **No revisar lo que se generó.** Aceptar cambios grandes sin leerlos es la forma más común de introducir bugs silenciosos.
- **Mezclar entorno de prueba y de producción.** Usar las mismas credenciales o la misma base de datos para probar y para usuarios reales expone a errores costosos.
- **No tener forma de volver atrás.** Sin control de versiones (git) con commits frecuentes, un cambio que rompe algo se vuelve mucho más difícil de revertir.

08 · Checklist final

- ✓ Definiste un plan de arquitectura antes de construir.
- ✓ Construiste por etapas verificables: base de datos, backend, frontend, integraciones.
- ✓ Probaste los flujos críticos, incluyendo casos límite.
- ✓ Hiciste un deploy real, con variables de entorno separadas de producción.
- ✓ Tenés un hábito de revisar logs y mantener el proyecto una vez lanzado.

FIN DE LA SERIE

Con los Niveles 1, 2 y 3 ya tenés el recorrido completo: crear algo desde cero sin saber programar, integrar IA a un flujo de trabajo real, y llevar un proyecto de punta a punta hasta producción. De acá en adelante, la mejor forma de seguir mejorando es construir proyectos propios, cada vez más ambiciosos.